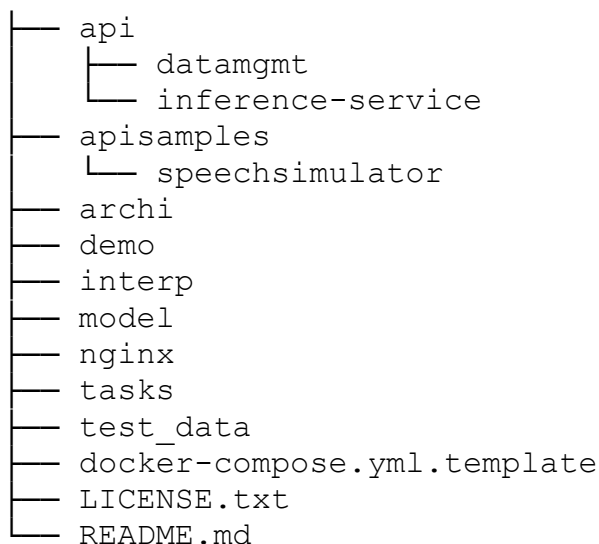


Развертывание программного комплекса

1 Описание репозитория

Репозиторий проекта расположен по адресу <https://github.com/dysata/dyslexia-child> и имеет следующую структуру:



Содержание репозитория:

- `model` – Python-модули, реализующие основную логику выявления признаков-маркеров в аудиозаписях (`dysmarkcccls_clean.py`) и классификации испытуемых по группам риска (`statdiag.py`),
- `api` – код двух сервисов:
 - `inference-service` – сервис анализа и классификации аудиозаписей: сетевой программный интерфейс (HTTP) доступа к Python-модулям анализа и классификации аудио, модельное приложение для скрининговой диагностики;
 - `datamgmt` – сервис сбора и разметки аудиозаписей для подготовки обучающего датасета: сетевой программный интерфейс (HTTP) доступа к хранилищам аудио и метаданных, веб-приложение сбора и разметки аудио, телеграм-бот для сбора аудио;
- `archi` – код для обучения модели,
- `demo` – Python-ноутбуки для
 - демонстрации выявления признаков-маркеров при анализе аудиозаписей,
 - демонстрации классификации испытуемых по группам риска,
 - демонстрации проверки годности аудиозаписей для анализа,
 - исследования времени анализа и потребления памяти,

- прототипы примеров приложений,
- interp – Python-ноутбук для объяснения (интерпретации) работы нейросети по выявлению признаков-маркеров,
- nginx – настройки Nginx-сервера, который обеспечивает единую точку доступа ко всем сервисам и шифрование внешнего трафика при взаимодействии с клиентскими приложениями,
- tasks – файлы с диагностическими заданиями,
- test_data – файлы аудиозаписей и текстовые расшифровки к ним, используемые Python-ноутбуками из каталога demo,
- docker-compose.yml.template – шаблон конфигурационного файла системы управления Docker-контейнерами Docker Compose,
- LICENSE.txt – MIT-лицензия,
- README.md – инструкции по использованию репозитория.

2 Требования к оборудованию и системному программному обеспечению

Минимальные требования к системе

- процессор: 64-битный процессор архитектуры x86-64, не ниже Intel Core i3 2-го поколения (ниже не тестировалось), однако для слабых процессоров с малым количеством ядер (относительно приведенной конфигурации и тестов выше) следует ожидать долгого времени обработки аудио;
- оперативная память не менее доступных 10Гб под модель и данные в связи с большим объемом модели;
- размер на диске не менее 50 Гб под файлы программного комплекса, объекты под управлением Docker (образы, контейнеры, тома, кэши в каталоге /var/lib/docker/), виртуальные окружения и библиотеки;
- операционная система Ubuntu 24.04.3 LTS

Параметры системы демонстрационного развертывания

Комфортная работа (время обработки аудио меньше времени записи аудио) обеспечивается на системах аналогичных системе, использованной для демонстрационного развертывания:

- Процессор: 2 x Intel(R) Xeon(R) CPU E5-2690 v2 @ 3.00GHz
- Оперативная память: 128 ГиБ

- Диск: TOSHIBA MK3261GS, 298 GiB
- ОС: Ubuntu 24.04.3 LTS

При этом в результате развертывания:

- каталог репозитория занимает 16GB,
- каталог `/var/lib/docker/` занимает 13GB.

Для исследования времени обработки аудио и потребления памяти разработан Python-ноутбук, который находится в репозитории проекта в файле `demo/benchmark_performance.ipynb`. По результатам испытаний на указанном оборудовании:

- время загрузки весов в память при наличии модели в кэше на диске: до 10 секунд,
- время обработки аудио: не более 0.7 секунд на секунду аудиозаписи,
- максимальное потребление памяти при анализе аудио: до 2.5 ГБ.

3 Система Docker-контейнеров

Рассматривается развертывание в серверном варианте на компьютере с установленной операционной системой Ubuntu 24.04.3 LTS.

Развертывание программного комплекса для использования представляет собой развертывания системы Docker-контейнеров с помощью системы управления контейнерами Docker Compose. В корне репозитория содержится шаблон конфигурационного файла Docker Compose: `docker-compose.yml.template`, который описывает систему контейнеров:

- **контейнер inference:** Реализует сервис анализа аудиозаписей, который предоставляет сетевой прикладной программный интерфейс (application programming interface, API) для выполнения запросов сторонних сетевых приложений об анализе аудиозаписей. Кроме программного интерфейса, реализует предоставление интерактивной документации (Swagger) по API и модельного приложения, которое демонстрирует применение API в разработке диагностических приложений. Сервис реализован с использованием фреймворка FastAPI, использует предварительно настраиваемое Python-окружение и нейросетевую модель, которая скачивается автоматически при развертывании системы с помощью сервиса Model Init Service (см. ниже). Путь к коду сервиса в репозитории проекта: `api/inference-service`.
- **контейнер datamgmt:** Реализует сервис, предоставляющий сетевой прикладной программный интерфейс для управления хранением аудиозаписей в хранилище Nextcloud и метаданных в хранилище CouchDB из веб-приложений. На основе этого

интерфейса сервис реализует также веб-приложение для записи и разметки аудио. Приложение предназначено для подготовки обучающих датасетов, просмотра имеющихся данных. Второе приложение, реализованное в рамках этого сервиса – Telegram-бот, который позволяет организовать сбор аудиозаписей через приложения Telegram. Сервис разработан на основе платформы Node.js.

- **контейнер games:** Реализует сервис демонстрационных приложений, который предоставляет ряд веб-приложений, так называемых “игр” (games). Аудиозаписи сохраняются в файловом хранилище сервиса и доступны для последующего анализа с помощью реализующих модель Python-модулей. Сервис разработан на основе фреймворка AIOnTTP и технологии WebRTC. Сервис позволяет совмещать аудиальную диагностику с записью нейрофизиологических сигналов – электроэнцефалографией (ЭЭГ). Синхронизация с ЭЭГ обеспечивается с помощью выставления TTL-меток на аппаратный синхропорт энцефалографа. Предполагается, что устройство должно быть подключено к компьютеру, на котором пользователь открывает веб-интерфейс приложения.

- **контейнер nextcloud:** Предоставляет пользователям облачное хранилище файлов, используется для хранения аудиозаписей, собираемых сервисом управления данными.

- **контейнер couchdb:** Предоставляет пользователям документоориентированную систему управления базами данных (СУБД) CouchDB, которая используется для хранения метаданных аудиозаписей, собираемых сервисом управления данными. Для целей разработки и администрирования организован доступ к пользовательскому веб-интерфейсу, который может быть отключен в настройках сервиса Nginx после того, как все необходимые настройки выполнены.

- **контейнер db:** Предоставляет пользователям реляционную СУБД MariaDB, которая используется штатно сервисом хранения файлов Nextcloud.

- **контейнер nginx:** Это развертывание сервера Nginx, реализующего единую точку доступа к остальным сервисам, маршрутизацию запросов, взаимодействие с клиентами по защищенному протоколу HTTPS (в то время как внутри между сервисами используются нешифрованные соединения), предоставление заглавной страницы системы сервисов (рисунок 1) с перечнем ссылок на остальные сервисы. Сервис способствует безопасности системы, скрывая внутреннее устройство системы от внешнего мира.

Dyslexia-child Services

Запись и разметка аудио

Подготовка размеченного датасета

REST API Demo

REST API documentation and services

Модельное приложение

Упражнения на произношение, оценка

Примеры приложений для совместной регистрации аудио и ЭЭГ

Упражнения на произношение, оценка

Nextcloud

Хранилище аудио для сервиса записи и разметки аудио

CouchDB

Хранилище метаданных для сервиса записи и разметки аудио

Рисунок 1 — Заглавная страница системы сервисов

- **контейнер model-init:** Реализует логику начальной настройки системы, запускается перед основными сервисами и выполняет следующую работу:

- проверяет наличие виртуального Python-окружения, при необходимости выполняет создание окружения и установку зависимостей для Python-программ, реализующих модель (в каталоге model/venv),
- проверяет наличие файлов модели, при необходимости загружает файлы обученной в проекте модели <https://huggingface.co/dysata/Wav2Vec2-Ru-Child> с сайта Hugging Face в локальный кэш (именованный том hf_cache).

После выполнения задач контейнер завершает работу. При необходимости повторной инициализации (при использовании в режиме разработки) необходимо перезапустить систему сервисов с помощью Docker Compose.

Сетевое взаимодействие. Все контейнеры подключены к одной внутренней виртуальной (Docker) сети, в которой сервисы общаются друг с другом по именам контейнеров (DNS resolution Docker). Например, сервис управления данными на Node.js обращается к CouchDB по адресу `http://couchdb:5984`. Никакие из сервисов

кроме nginx не имеют портов открытых в Internet. Доступ к ним осуществляется проксированием через Nginx. Контейнер Nginx имеет проброшенные порты 80 и 443 на хост. Маршрутизация запросов к сервисам реализована на основе пути относительно адреса домена (authority):

- /datamgmt – доступ к сервису записи и разметки аудио,
- /inference – доступ к сервису анализа аудиозаписей (и модельному приложению на его основе),
- /games – доступ к сервису демонстрационных приложений,
- /nextcloud – доступ к хранилищу Nextcloud,
- /couchdb – доступ к документной базе данных CouchDB.

Монтируемые хранилища. Для обеспечения сохранности данных при перезапуске и обновлении контейнеров используется система именованных томов (Named Volumes), анонимных томов (Anonymous Volume) и монтируемых каталогов (Bind Mounts).

Данные CouchDB, MariaDB, Nextcloud хранятся в управляемых Docker томах: couchdb_data, db, nextcloud. Это гарантирует, что данные не будут потеряны при удалении или пересоздании контейнеров этих сервисов. Также организован общий том для хранения (кэш) параметров модели, загружаемых с Hugging Face: hf_cache, он монтируется в контейнер инициализации (для записи) и контейнер анализа аудиозаписей (для чтения). Сервис анализа аудиозаписей также использует том inference_data, а сервис демонстрационных приложений – games_storage. Исходный код сервисов и скрипты моделей монтируются напрямую из файловой системы хоста (см. разделы volumes сервисов в docker-compose.yml.template). Это позволяет обновлять логику без пересборки образов (в режиме разработки). Логика инициализации создает папку с виртуальным окружением Python в папке model, в этой же папке model лежат Python-модули, она монтируется к контейнерам, использующим модель. Сервис Nginx монтирует папку nginx, в которой расположены настройки сервиса и где нужно размещать файлы SSL-ключей для организации предоставления доступа к системе сервисов по шифрованному протоколу HTTPS. Модули Node.js, используемые сервисом datamgmt, устанавливаются в анонимный контейнер, который монтируется в /app/node_modules datamgmt, что позволяет избежать повторной установки модулей без необходимости при пересборке контейнера.

4 Начальная настройка

Переменные окружения в Docker Compose: Все чувствительные параметры (пароли, токены, пути) задаются в файле конфигурации `docker-compose.yml` и передаются в контейнеры через переменные окружения (environment variables), что позволяет легко менять конфигурацию без изменения кода или образов. Необходимо создать файл на основе шаблона, выполнив в корневом каталоге репозитория команду копирования:

```
cp docker-compose.yml.template docker-compose.yml
```

Далее необходимо отредактировать файл с помощью любого редактора. Требуется уточнить следующие параметры (задаются строками без кавычек):

контейнер couchdb:

COUCHDB_PASSWORD: пароль администратора CouchDB, задается произвольно

контейнер datamgmt:

BASE_URL: полный URL вида `https://<домен>/datamgmt`

NEXTCLOUD_PASSWORD: пароль пользователя nextclouduser сервиса Nextcloud (см. далее Начальная настройка), задается произвольно

PASSWORD: пароль сервиса сбора и разметки аудио, задается произвольно

COUCHDB_PASSWORD: пароль администратора CouchDB (тот же, что выше)

TELEGRAM_TOKEN: API-токен телеграм-бота (см. порядок получения <https://core.telegram.org/bots/tutorial>)

TELEGRAM_PROXY_URL: HTTP-прокси сервер для работы телеграм бота, строка вида `http://<имя пользователя прокси>:<пароль>@<ip или FQDN>:<порт>`

контейнер nextcloud:

OVERWRITEHOST: полное доменное имя сервера развертывания (FQDN)

OVERWRITECLIURL: строка вида `https://<FQDN>/nextcloud`

контейнер inference:

API_KEY: ключ доступа к API сервиса, задается произвольно

Настройка CouchDB: После запуска системы контейнеров необходимо зайти в панель веб-интерфейса CouchDB `https://<полное доменное имя сервера развертывания>/couchdb/_utils/#login` (используются логин и пароль, заданные в `docker-compose.yml`, см. выше), создать базу данных `dyslexia` (рисунок 2) и представление (view) (рисунок 3) с названием `human_tasks`:

```
function (doc) {
```

```

if(doc.human_task) {
  emit(doc.human_task.task_series_id, doc);
}
}

```

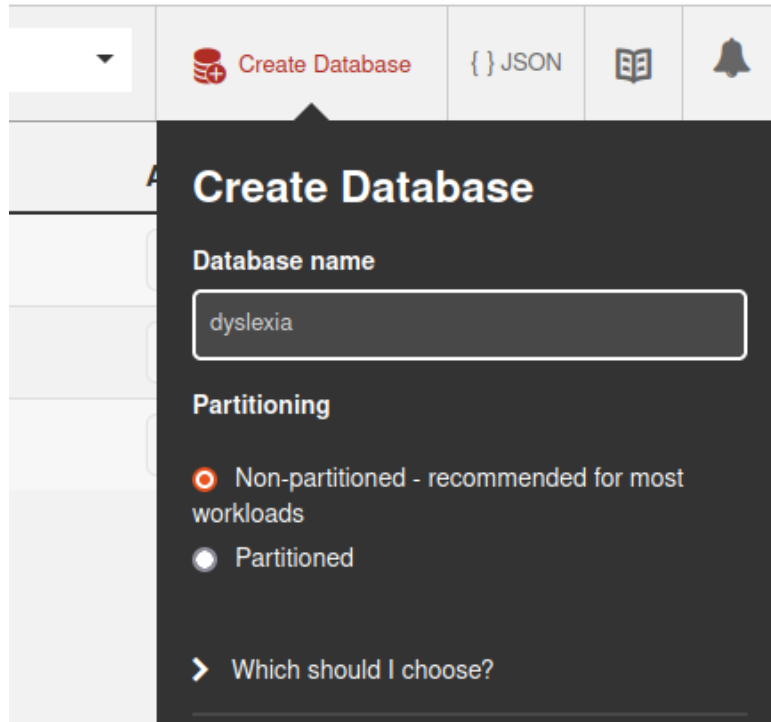


Рисунок 2 — Создание базы данных dyslexia в CouchDB

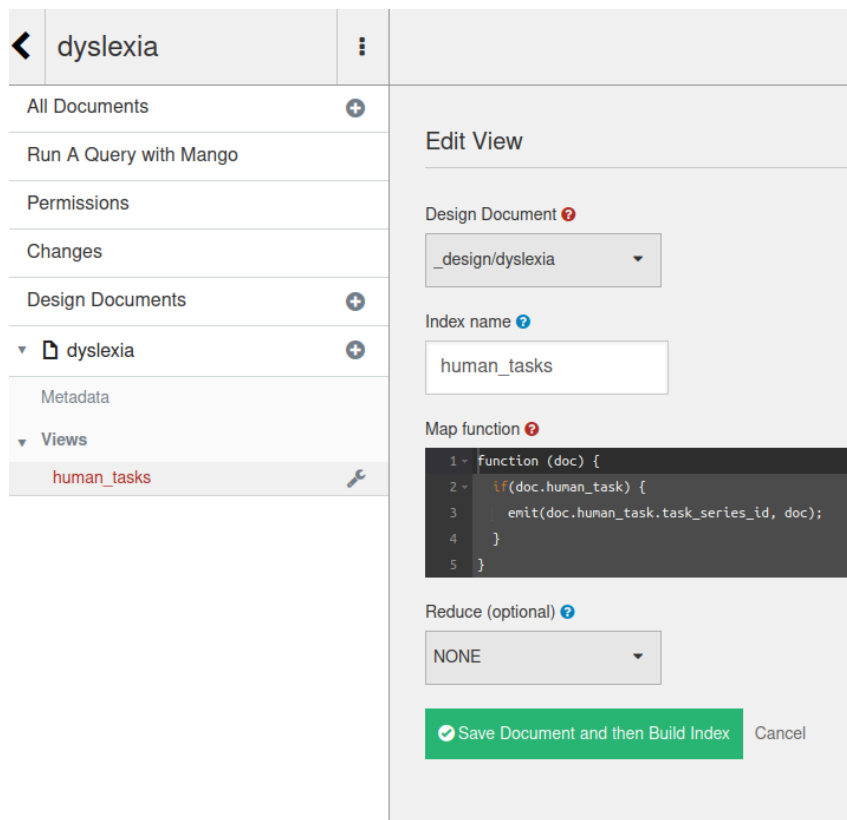


Рисунок 3 — Создание представления (view) human_tasks в базе данных в CouchDB

Настройка Nextcloud: Нужно открыть страницу начальной настройки Nextcloud <https://<полное доменное имя сервера развертывания>/nextcloud>. Там создать пользователя с логином nextclouduser и паролем, заданным в docker-compose.yml, см. выше. Пройти аутентификацию с этими данными. Создать папку dyslexia.

Настройка Nginx: Для организации работы сервера Nginx с внешними клиентами по шифрованному HTTPS протоколу необходимо поместить файлы ключа (privkey.pem) и сертификата (fullchain.pem) в каталог nginx/ssl. Организацию получения необходимых файлов можно организовать с помощью сервиса Let's Encrypt или аналогичного (см. документацию этих сервисов, <https://letsencrypt.org/ru/>) или создать т.н. самоподписанный сертификат, например (подробнее см. документацию openssl <https://docs.openssl.org/master/man1/openssl>):

```
openssl req -x509 -newkey rsa:4096 -keyout privkey.pem -out fullchain.pem -sha256 -days 365 -nodes
```

Должны быть установлены владелец и права для этих файлов:

```
sudo chown root:root fullchain.pem privkey.pem
```

```
sudo chmod 644 cert.pem
```

```
sudo chmod 600 key.pem
```

5 Процесс развертывания

Для старта системы контейнеров в корневом каталоге репозитория следует выполнить команду, которая собирает Docker-образы, создает и запускает контейнеры (при необходимости: для вариантов, в т.ч. поэтапного выполнения – см. документацию Docker Compose):

```
docker-compose up --build -d --force-recreate
```

В результате происходят следующие действия:

1. Инициализация: При запуске команды docker-compose up первым стартует сервис model-init. Он проверяет наличие Python-окружения, моделей, при необходимости создает виртуальное окружение, устанавливает зависимости и скачивает модели в общий том hf_cache. Основные сервисы ждут его успешного завершения.
2. Старт инфраструктуры: Поднимаются контейнеры баз данных. При первом запуске они инициализируют файлы данных в томах.

3. Старт приложений: Запускаются сервисы `datamgmt`, `inference`, `games`. Они подключаются к готовым томам с моделями и данными.

3. Маршрутизация: `Nginx` считывает конфигурацию и начинает принимать внешний трафик, проксируя его внутрь сети.

6 Проверка работы программ и сервисов

Приводится несколько примеров, позволяющих проверить работу развернутого программного комплекса.

6.1 Python-реализация модели

Python-реализация модели имеет два основных модуля (папка `model` в репозитории):

- `dysmarkccls_clean.py` – основной модуль, который может подключаться к пользовательским программам или использоваться как самостоятельная программа для выявления маркеров-признаков в аудиозаписях,
- `stat.py` – модуль для классификации аудиозаписей по группам риска на основе данных о маркерах-признаках, полученных в результате применения модуля `dysmarkccls_clean.py`.

Каталог содержит ряд вспомогательных модулей и программу `download_model.py`, которая используется для предварительного скачивания модели с сайта Hugging Face в локальный кэш при развертывании системы.

Для использования этих программ в пользовательском режиме требуется настроить виртуальное окружение. Находясь в каталоге с репозиторием кода проекта (проведено тестирование с Python версии 3.12.3):

```
sudo apt update && sudo apt upgrade -y
sudo apt install python3 python3-venv python3-pip -y
cd model
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

После завершения работы можно деактивировать и удалить окружение, если оно более не потребуется:

```
deactivate
```

```
rm -Rf .venv
```

Когда виртуальное окружение настроено, можно исследовать работу программ.

Примеры команд:

1. Выявление признаков-маркеров в аудиозаписях:

```
python dysmarkcls_clean.py <путь к файлу с текстом> <путь к файлу с аудио>
```

Примеры текстовых и аудиофайлов можно найти в каталоге test_data

Пример вывода:

```
{
  "triples": [
    ["в", "в", "0"],
    ["н", "в", "7"],
    ["а", "а", "0"],
    ["ш", "ш", "0"]
  ],
  "quality": "good"
}
```

2. Классификация по группам риска:

```
cat <файл_результатов.json> | python3 stat.py --stdin
```

Пример входного файла (набор ответов предыдущей команды):

```
{
  "analyses": [
    {
      "triples": [["м", "м", "0"], ["а", "а", "0"], ["м", "н", "7"]]
    },
    {
      "triples": [["к", "к", "0"], ["о", "о", "0"], ["т", "", "В"]]
    }
  ]
}
```

Пример вывода:

```
{
  "total_phonemes": 450,
  "total_markers": 65,
  "marker_statistics": {
    "К": 12,
    "7": 5,
    "В": 8,
    "С": 40
  },
  "norma": 15,
  "risk_group": "worry about (имеются нарушения, требуется наблюдение)"
}
```

6.2 Набор демонстрационных Python-ноутбуков

В каталоге demo репозитория расположено 4 Python-ноутбука:

- test_quality_checks.ipynb - демонстрирует анализ годности аудиозаписей для исследования,
- test_marking.ipynb – демонстрирует анализ аудиозаписей на выявление признаков-маркеров,
- classification_test.ipynb – демонстрирует классификацию результатов анализа по группам риска,
- benchmark_performance.ipynb – исследует время анализа аудиозаписи и потребление памяти.

Для работы с этими ноутбуками нужно настроить виртуальное окружение (можно переиспользовать окружение из model, см. предыдущий раздел). Для настройки окружения в каталоге с репозиторием кода выполняются команды:

```
sudo apt update && sudo apt upgrade -y
sudo apt install python3 python3-venv python3-pip -y
cd demo
python3 -m venv jupyter_env
source jupyter_env/bin/activate
pip install -r requirements.txt
```

Далее нужно запустить сервис для работы с ноутбуком:

```
jupyter notebook <имя Python-ноутбука>.ipynb
```

Как правило, в результате запускается автоматически браузер и ноутбук становится доступным для работы в открытом окне браузера. Если этого не произошло, обратите внимание на то, что команда распечатывает в терминале ссылку вида:

```
http://localhost:8888/tree?token=<строка токена>
```

Её следует открыть в браузере и далее выбрать в браузере один из Python-ноутбуков: файлы с расширением interp.ipynb.

После работы с Python-ноутбуками для закрытия сервиса нажмите комбинацию ctrl+c в терминале, где был запущен сервис. Для деактивации виртуальной среды выполните команду

```
deactivate
```

Если виртуальная среда больше не требуется, удалите каталог с ее настройками:

```
rm -Rf jupyter_env
```

6.3 Веб-приложения и программные интерфейсы

Приводится список сервисов и URL и краткое описание (FQDN – полное доменное имя сервера развертывания):

- <https://<FQDN>/datamgmt/> – Веб-приложение сбора и разметки аудио, тестовый пользователь – user, пароль был указан в конфигурации docker-compose.yml
- <https://<FQDN>/inference/v1/docs> – Интерактивная документация (Swagger) по программному интерфейсу сервиса анализа аудиозаписей
- <https://<FQDN>/inference/static/demo/index.html> – Модельное приложение скрининговой диагностики, демонстрирующее использование программного интерфейса.
- <https://<FQDN>/games/> – Сервис с демонстрационными приложениями совместной регистрации аудиозаписей и энцефалограмм.
- <https://<FQDN>/nextcloud/> – Пользовательский интерфейс системы облачного хранилища файлов Nextcloud (используется для хранения аудиозаписей сервисом сбора и разметки аудио).
- https://<FQDN>/couchdb/_utils/ – Пользовательский интерфейс СУБД CouchDB (используется для хранения метаданных аудиозаписей сервисом сбора и разметки аудио).